

# Compiling FFI-heavy OCaml to WebAssembly

An Experience Report on MOPSA

---

Reda Boudrouss

OCaml Workshop 2026 · August 24, Paris

Sorbonne University



## The case file

---

# The project

**MOPSA:** a static analyzer by abstract interpretation, for C & Python

- written *mostly* in OCaml...
- ...on GMP, MPFR, Zarith, Apron, LLVM/Clang. All bound through the FFI

**Goal:** run it entirely in the browser.  
No server.

It works today:

<https://mopsawasm.rboud.com/>

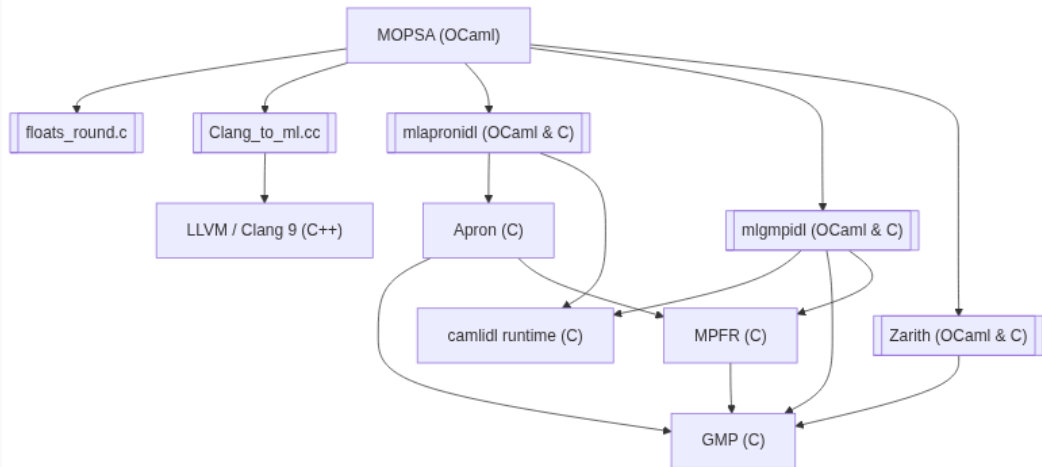
The screenshot displays the MOPSA web interface. On the left, a file explorer shows a project structure with files like `example.c`, `main.c`, `utils.c`, `example.py`, `main.py`, `mytest.c`, `universal`, and `example.u`. The main panel shows the source code of `example.c`, which includes standard headers, a `sign(int x)` function, a `classify(int a, int b)` function, and a `main()` function. The right sidebar contains a summary of the analysis results:

| SUMMARY          |            |
|------------------|------------|
| 147 TOTAL        | 148 ERRS   |
| 1 WARNINGS       | 0 ERRORS   |
| 146/147 ANALYZED | 4.09s TIME |

Below the summary, a 'CHECKS' section shows various checks with their status (e.g., 'Integer overflow' is marked as 'AT -55'). A 'WARNINGS' section lists specific warnings, such as 'warning: expected expression' and 'warning: expected '}'.

At the bottom, a '1 ISSUE' section details an 'Integer overflow' error: 'Integer overflow: 'int32\_t' has value 1,3247463645 that is larger than the range of 'int32\_t'.'

# The dependency stack



Every box ships C or C++ · double-bordered boxes use the FFI to build or read OCaml values

## The FFI goes both ways

Clang\_to\_ml.cc 5000 lines, both worlds in one translation unit:

```
#include "clang/AST/RecursiveASTVisitor.h"
#include "clang/Frontend/CompilerInstance.h"

#include <caml/mlvalues.h>
#include <caml/alloc.h>
#include <caml/memory.h>
```

Each Clang AST node triggers `caml_alloc + Store_field`: allocation *inside the OCaml GC heap, from C++*.

CamlIDL generates ~655 more such stubs for Apron & GMP.

Every call site bakes in what an OCaml value is, byte for byte, in memory.

## The usual suspects don't fit

`js_of_ocaml`, `wasm_of_ocaml`, `wasocaml`: they compile the OCaml and **leave the C/C++ behind**.

- fine for pure OCaml, or with a JS reimplementation (`zarith_stubs_js`)
- here: rewrite 5000 lines of `Clang_to_ml.cc` *and* 655 generated stubs? no.

A structural problem: **what do the FFI stubs link against?** `caml_alloc`, `caml_callback`, ... exist in no translated module.

And exposing them wouldn't help: WasmGC backends move OCaml values *out of linear memory*, so a C stub reading a tag by pointer **cannot reach them at all**.

## The solution

---



# Ship the runtime itself

The missing `caml_*` symbols are defined in the **the OCaml runtime**.

1. compile MOPSA to **bytecode**
2. compile the **runtime** + every native dep to wasm (Emscripten)
3. **link statically** into one module
4. **interpret** `mopsa.bc` on top

|                             |
|-----------------------------|
| mopsa.bc (interpreted)      |
| ocamlrun.wasm · one module  |
| libcamlrun + prims          |
| GMP · MPFR · Apron · Zarith |
| camlidl rt · LLVM/Clang     |
| Clang_to_ml                 |

One toolchain (emcc), one memory model, no per-binding glue.

## Binding 1435 externals without dlopen

Bytecode never calls C directly: C\_CALLn <index>, resolved at startup by dlopen/dlsym.

But read the interpreter's lookup path first:

```
static c_primitive
lookup_primitive(char *name) {
    /* 1. static table, FIRST */
    for (i = 0; names[i]; i++)
        if (!strcmp(name, names[i]))
            return builtin_cprim[i];
    /* 2. only then: dlsym() */
```

```
/* generated prims.c */
extern value caml_array_get();
extern value unix_read();
/* ... 1435 lines ... */
c_primitive caml_builtin_cprim[]
= { caml_array_get,
    unix_read, /*...*/ 0 };
```

Scan every C/C++ source, generate a **superset** prims.c (runtime core, unix (131), str, bigarray, **655 camlidl** Apron/GMP stubs), disable the dlopen branch.

## The build montage

| library         | version | manual trick                                    |
|-----------------|---------|-------------------------------------------------|
| OCaml runtime   | 4.14.2  | sak is a <i>host</i> tool: rebuild with real cc |
| GMP             | 6.1.2   | --disable-assembly --host=none                  |
| MPFR            | 4.2.2   | n/a                                             |
| CamlIDL runtime | latest  | 3 files, compiles as-is                         |
| Apron           | lastest | n/a                                             |
| LLVM/Clang      | 9       | two-stage: native tblgen (gcc-11),<br>then emcc |
| Clang_to_ml.cc  | n/a     | Clang <i>and</i> caml headers                   |

OCaml 4.14, not 5: the target is wasm32, and OCaml 5 dropped 32-bit.  
Several dependencies compiled with **no patch at all**.

## The final link

```
emcc ... -o ocamlrun.js \  
  --preload-file mopsa.bc@/build/mopsa.bc \  
  libs/*.a \  
  -s ERROR_ON_UNDEFINED_SYMBOLS=1 \  
  prims.o libcamlrun.a
```

- **everything statically linked**: one self-contained 15 MB .wasm
- ERROR\_ON\_UNDEFINED\_SYMBOLS=1: a missing primitive fails the *link*, loudly, not the browser at runtime
- mopsa.bc is *preloaded*, not linked: it's data, interpreted

## The final link

```
emcc ... -o ocamlrun.js \  
  --preload-file mopsa.bc@/build/mopsa.bc \  
  libs/*.a \  
  -s ERROR_ON_UNDEFINED_SYMBOLS=1 \  
  prims.o libcamlrun.a
```

- **everything statically linked**: one self-contained 15 MB .wasm
- ERROR\_ON\_UNDEFINED\_SYMBOLS=1: a missing primitive fails the *link*, loudly, not the browser at runtime
- mopsa.bc is *preloaded*, not linked: it's data, interpreted

Every symbol resolved. It links. It's self-contained.

and at runtime, in the browser

---

**Uncaught RuntimeError:  
index out of bounds**

## The investigation

---

## Exhibit A: what an OCaml value is



The trap traces to reading a block's *tag* (OCaml 4.14, little-endian):

```
#define Tag_val(val) (((unsigned char *) (val)) [-sizeof(value)])
```

Easy hypotheses, eliminated:

- `sizeof(value)` miscompiled? No: a compile-time constant, always 4
- a 64/32-bit mismatch? No: the ILP32 config is perfectly consistent



## Exhibit A: `-sizeof(value)` is not `-4`

`sizeof` has type `size_t`, which is **unsigned**:

`-sizeof(value) = -(size_t)4 = 0xFFFFFFFFFC` (not `-4!`)

**Native 32-bit:** `p + 0xFFFFFFFFFC` is a wrapping 32-bit add  $\rightarrow$  exactly `p - 4`.  
Upstream OCaml works everywhere, *by wraparound*.

**wasm32:** two ways to add an offset:

- an explicit `i32.add` wraps modulo  $2^{32}$ : fine
- the **static offset=N** immediate of a load is an *unsigned* `u32`, bounds-checked on the full untruncated value: **no wraparound**

LLVM may fold any *non-negative* constant into `offset=N`. `0xFFFFFFFFFC` qualifies  $\rightarrow$  effective address  $\approx 4$  GiB  $\rightarrow$  the bounds check **traps**.

## Exhibit A: the only difference is signedness

```
;; ((unsigned char *) v)
;;      [-sizeof(value)]
(func $tag_old (param i32)
              (result i32)
```

```
    local.get 0
    i32.load8_u
    offset=4294967292)
```

folded into the load → **traps**

```
;; ((unsigned char *) v)
;;      [-(int)sizeof(value)]
(func $tag_fixed (param i32)
              (result i32)
```

```
    local.get 0
    i32.const -4
    i32.add
    i32.load8_u)
```

wrapping add →  $p - 4$ , fine

The fix: cast to *signed* before negating, since a negative displacement can't be folded into `offset=N`. (re-verified with the project's emcc 4.0.22)

The fold is backend luck: wasi-sdk clang 18 declines it. The fix removes the bet entirely.

## Exhibit B: va\_list changes shape

Porting to wasm32 also retargeted the *embedded* Clang to 32-bit: the sources MOPSA parses are now typed for a 32-bit ABI.

- **x86-64**: `va_list` is an array → decays to a pointer, handled
- **32-bit**: a scalar `void *`, passed *by reference* to `__builtin_va_start` → an `LValueReferenceType`, a case the type translator never needed before

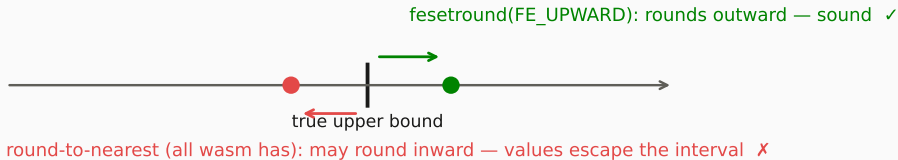
unhandled type: `lvalue_ref(__builtin_va_list=void*)`

This broke parsing of the CPython stabs, and with them **all cross C/Python analysis**.  
The fix (a reference is ABI-equivalent to a pointer):

```
| C.LValueReferenceType tq -> T_pointer (type_qual range tq), no_qual
```

## Exhibit C: nobody controls the rounding

Wasm has **no FPU rounding-mode control**: everything rounds to nearest, `fesetround` is a no-op. Not a 32-bit quirk: **a property of wasm itself**.



- **Apron**: compile every domain with `NUM_MPQ`, so bounds are computed in exact GMP rationals and `fesetround` is never needed. Stub the FPU probe (`-Wl,--wrap=ap_fpu_init`). *Residual*: floats reaching Apron's API.
- **MOPSA's own `floats_round.c`**: *no satisfying fix yet*; widening keeps the analysis sound but coarse.

## From module to app

---

## A fresh instance per analysis

The OCaml runtime keeps its state **global** and is not re-entrant, and `mopsa.bc` ends with `exit`. There is no clean way back.

- keep one instance, suspend around I/O with **Asyncify**? incompatible: OCaml exceptions are `setjmp/longjmp`, and both rewrite the stack
- **a fresh instance for every analysis** (`MODULARIZE=1`)

The web worker fetches + `compileStreamings` the module **once**, then each analysis only *re-instantiates* the already-compiled module.

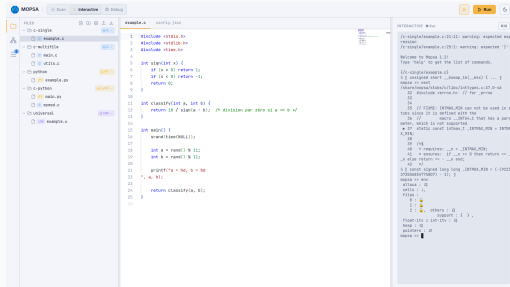
Full module setup (compile, instantiate, unpack the FS data) is **63 ms** under Node, re-instantiation alone is cheaper still.

(Browser: 641 ms for the one-shot first start, fetch included.)

# The interactive debugger is a frozen Worker

Interactive REPL & DAP debugger:  
long-lived runs that **block reading stdin**,  
inside a Worker that can't service  
postMessage while frozen.

The only web primitive that fits:  
**SharedArrayBuffer + Atomics.wait**  
(needs cross-origin isolation) Cross Origin  
Embedder Policy and Cross Origin Opener  
Policy HTTP headers

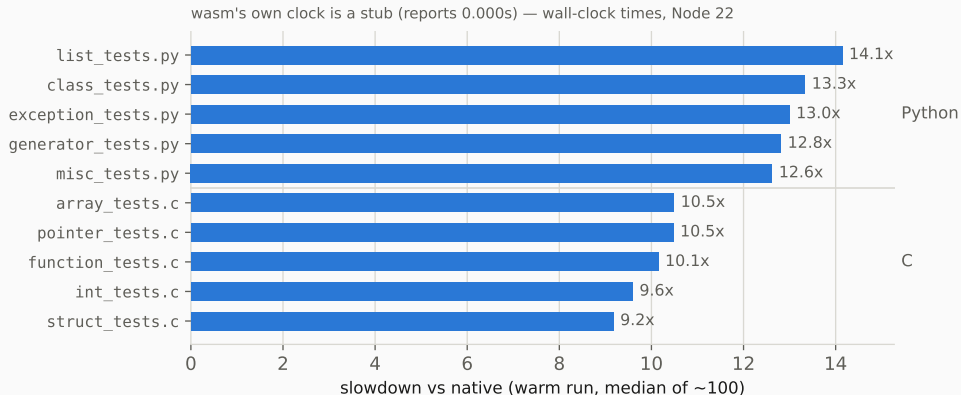


## The receipts

---



# The cost of interpretation



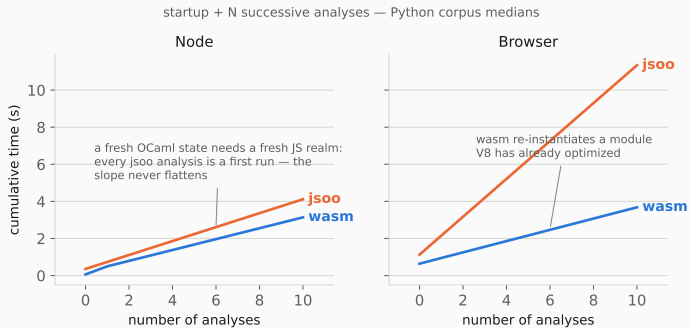
Interpreted bytecode on wasm vs native compiled code, V8 steady state. `int_tests.c`

40 ms → 384 ms

`struct_tests.c` 411 ms → 3.77 s

# Where js\_of\_ocaml can't follow

the baseline is **Try-MOPSA**, a scaled-down jsso build: no C frontend, Apron replaced by VPL (pure OCaml).



startup 63 vs 359 ms (Node), 641 ms vs 1.13 s (browser)

repeated Python: wasm  $\sim 1.3\times$  (Node) /  $\sim 3.4\times$  (browser) ahead

optimistic for jsso: its 22 MB bundle parse is charged only once

## Closing

---

## What wasm32 actually taught us

*was32 is almost a 32-bit target; the differences hide in the corners.*

| looked like             | actually was                            | failure mode       |
|-------------------------|-----------------------------------------|--------------------|
| bad index / codegen bug | unsigned negation folded into offset=N  | loud trap          |
| a Clang parser bug      | 32-bit ABL: va_list passed by reference | loud error         |
| <i>nothing at all</i>   | no rounding-mode control in wasm        | silent unsoundness |

Two failures were loud, the third silent: **the silent one is the reason to talk about this.**

# The community question

None of this is specific to MOPSA, and most of it is mechanical:

---

step

---

compile project to bytecode

mechanical

compile runtime + deps with emscripten

mostly mechanical (several deps  
unpatched)

generate `prims.c`, link statically

systematic

genuinely native pieces (LLVM, FFI  
hazards)

needs care

---

**Worth a shared effort?** A dune/opam build target: whole project + native stack → wasm, from an opam switch, at least until a WasmGC backend can interoperate with Emscripten-compiled C. (OCaml 5? `Tag_val` is fixed in 5.x, but 5 dropped 32-bit.)

## Try it & thanks

**Demo** (works right now):

<https://mopsawasm.rboud.com/>

**Code:**

<https://github.com/rboudrouss/mopsa-emcc>

**Antoine Miné:** guidance through MOPSA and support throughout ·

**Raphaël Monat:** Try-MOPSA, the jsoc/VPL baseline ·

**Vincent Chan:** ocaml-wasm configure tweaks & Unix stubs ·

**Ben Smith (binji):** LLVM-to-wasm fork and notes



**I'm on the job market**, looking for a **CIFRE PhD** host company (French industrial PhD, on static analysis)

## Backup

---

## Full benchmark synthesis (~100 reps, medians)

| target           | files | inst.  | cold run | hot run | Mopsa     | RSS MB |
|------------------|-------|--------|----------|---------|-----------|--------|
|                  |       |        |          |         | self-time |        |
| native           | 14    | n/a    | 23 ms    | n/a     | 23 ms     | 70     |
| wasm-node        | 14    | 63 ms  | 476 ms   | 294 ms  | n/a*      | 193    |
| jsoo-node        | 9     | 359 ms | 371 ms   | n/a**   | 364 ms    | 200    |
| wasm-<br>browser | 14    | 641 ms | 356 ms   | 304 ms  | n/a*      | 35***  |
| jsoo-browser     | 9     | 1.13 s | 1.01 s   | 1.00 s  | 298 ms    | 35***  |

\* the wasm runtime clock is a stub (always 0.000 s) → wall-clock used · \*\* jsoo-node cannot re-enter a fresh OCaml state in-process · \*\*\* browser RSS ≈ main-thread heap only, worker not counted · inst. = compile + instantiate + unpack .data (one-shot first start in the browser) · browser “cold” is quasi-hot: the page is shared across files. AMD Ryzen 7 1700X, Node 20 (V8 10.4), Chrome 116 (Docker)



## Backup: the sak catch

OCaml 4.14 introduced `runtime/sak`, a *host* tool that encodes the `stdlib` path as a C string. Under `emconfigure` it silently compiles to a `.wasm` that can't run on the host: the path stays empty, failure surfaces at runtime.

```
CFLAGS="$CFLAGS" $(EMCONFIGURE) ./configure \
    --disable-native-compiler --disable-systhreads ...
rm -f runtime/sak runtime/sak.o runtime/sak.wasm
cc -c -o runtime/sak.o runtime/sak.c && cc -o runtime/sak runtime/sak.o
touch runtime/sak.o runtime/sak
CFLAGS="$CFLAGS" $(MAKE) -C runtime libcamlrun.a
```

`runtime/` also provides the `<caml/*.h>` headers, so every stub is compiled against the exact runtime that executes it.

## Backup: LLVM/Clang 9, two-stage build

**Stage 1 (native):** only `llvm-tblgen` & `clang-tblgen`, with `gcc-11` (LLVM 9 no longer compiles with recent `gcc/clang`).

**Stage 2 (wasm):** `cmake` with the Emscripten toolchain,  
`-DLLVM_TABLEGEN/-DCLANG_TABLEGEN` pointing at stage 1.

```
ninja clangFrontend clangParse clangAST clangLex clangBasic  
      clangSema clangDriver ... LLVMSupport LLVMCore ...
```

Only the parsing frontend: no codegen backends, no optimizers. Clang's resource headers (`stddef.h`, ...) installed then preloaded into the virtual FS;  
`-DCLANGRESOURCE="/clang-headers"`.

## Backup: GMP, MPFR, CamlIDL

- **GMP 6.1.2:** `emconfigure ./configure --disable-assembly --host=none`
- **MPFR 4.2.2:** `touch aclocal.m4 configure (skip autoconf re-run),`  
`--with-gmp=$(INSTALL_DIR)`
- versions pinned to the pair known to compile cleanly with Emscripten (via a Stack Overflow answer; software archaeology counts)
- **CamlIDL:** runs at *build time* on the host; only its 3-file runtime (`idlalloc.c`, `comintf.c`, `comerror.c`) is compiled to wasm
- **Apron:** each domain (box, oct, polka) as its own archive, `-DNUM_MPQ`

## Backup: the va\_list patch in full

*(\* References are ABI-equivalent to pointers; model them as such. These surface via builtins such as \_\_builtin\_va\_start, whose va\_list argument is passed by reference when va\_list is a scalar (a void pointer) on 32-bit targets, unlike x86-64 where it is an array that decays to a pointer. \*)*

```
| C.LValueReferenceType tq -> T_pointer (type_qual range tq), no_qual  
| C.RValueReferenceType tq -> T_pointer (type_qual range tq), no_qual
```

Trigger: share/mopsa/stubs/cpython/Python.c, PyErr\_Format's va\_start; it blocked all cross C/Python analysis on 32-bit.

## Backup: versions & availability

- OCaml **4.14.2** (bytecode runtime) · LLVM/Clang **9** · GMP **6.1.2** · MPFR **4.2.2** · emcc **4.0.22**
- artifacts: `ocamlrun.wasm` 15 MB · `ocamlrun.data` 21 MB (virtual FS: `mopsa.bc`, Clang resource headers, linux32 headers, `share/mopsa`)
- fully client-side: <https://mopsawasm.rboud.com/>
- sources: <https://github.com/rboudrouss/mopsa-emcc>
- OCaml 5: `Tag_val` fixed upstream in latest 5.x; early experiment suggests the remaining wasm-hostile behavior can be disabled, but OCaml 5 dropped 32-bit support, so `wasm32` needs more than that